Dockerized Workout MERN Application – Project Documentation

1. Project Overview

This project is a Dockerized MERN (MongoDB, Express, React, Node.js) application. The purpose is to containerize all services (frontend, backend, and database) to ensure consistent development and deployment using Docker and Docker Compose.

---

2. Technologies Used

- Node.js (v20)
- React (Vite / Development Server)
- MongoDB
- Docker
- Docker Compose

---

3. Architecture Overview

The application consists of three services:

- Backend → Node.js API (port 3000)
- Frontend → React app (port 5173)
- Database → MongoDB (port 27017)

All services communicate through a custom Docker bridge network:

Network Name: workout-app-network

---

4. Backend Docker Configuration

```
FROM node:20

WORKDIR /app

COPY package*.json ./
```

```
RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "run", "start:dev"]
```

Explanation:

- Uses Node.js 20
- Installs dependencies
- Runs backend in development mode
- Exposes API on port 3000

---

5. Frontend Docker Configuration

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 5173

CMD ["npm", "run", "dev", "--", "--host"]
```

Explanation:

- Lightweight Alpine Node image
- Runs React/Vite development server
- Exposes frontend on port 5173

---

6. Docker Compose Configuration

```
services:
  backend:
    build: ./backend
    ports:
      - "3000:3000"
    env_file:
      - ./backend/.env
    depends_on:
      - mongodb
    networks:
      - workout-app-network

  frontend:
    build: ./frontend
    ports:
      - "5173:5173"
    networks:
      - workout-app-network

  mongodb:
    image: mongo
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db
    networks:
      - workout-app-network

volumes:
  mongo_data:

networks:
  workout-app-network:
    driver: bridge
```

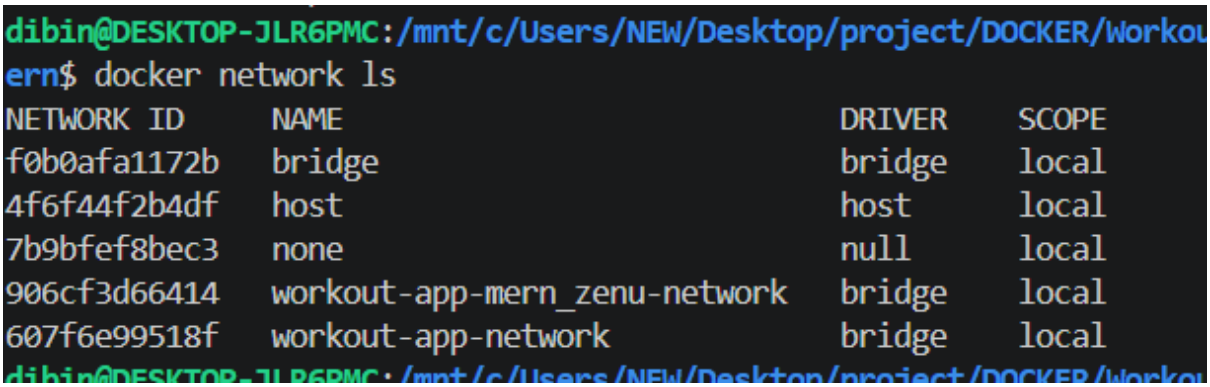
---

7. Networking

All containers communicate using a shared bridge network:

- Network Name: workout-app-network

- Driver: bridge
- Purpose: Enables inter-service communication using container names



---

8. Persistent Storage

MongoDB uses Docker volume:

- Volume: mongo_data
- Purpose: Keeps database data persistent across restarts

---

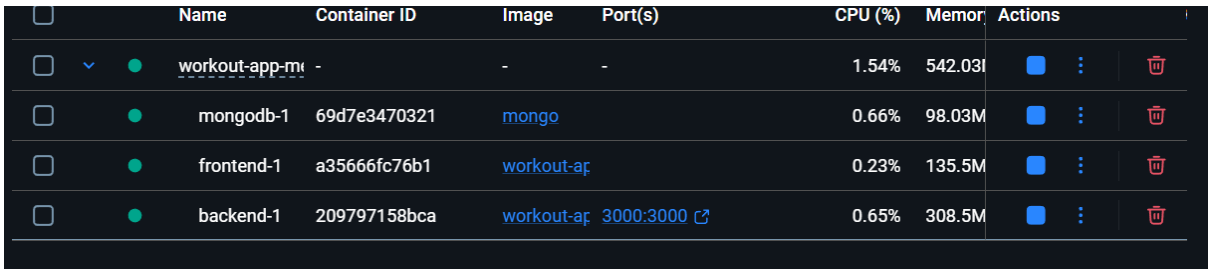
9. Deployment Commands

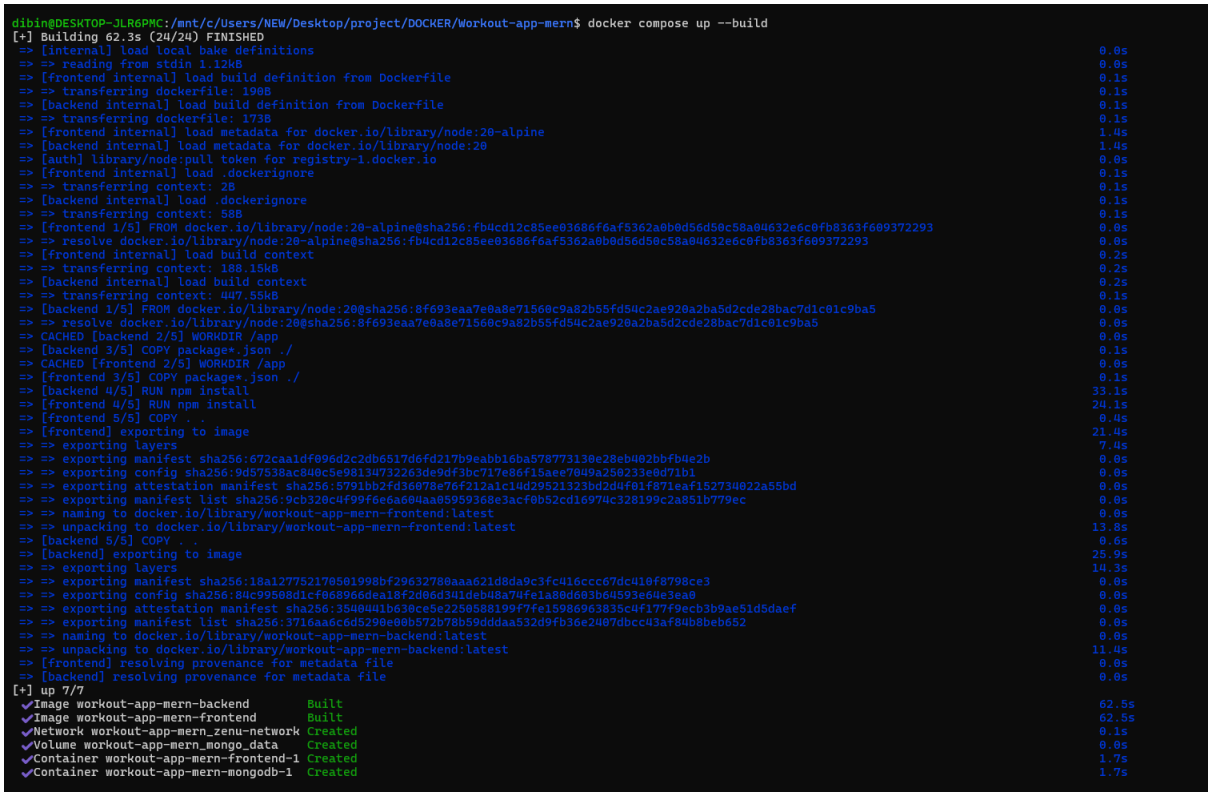
Start project:

docker compose up --build

Stop project:

docker compose down





---

10. Conclusion

This project demonstrates a complete Dockerized MERN stack setup with isolated services, persistent storage, and a shared network (workout-app-network) ensuring portability and scalability.